

SIMATS ENGINEERING

Department of Computer Science Engineering

Brute Force Geometric Algorithms: A Comparative Study of Closest-Pair Detection and Convex Hull Construction

Student Name	G Uday Siva Sai
Register / Student ID	192425259
Section / Batch	AI&ML-2024
Programme	B.E. Computer Science Engineering — III Year
Course Code & Name	CSA06 — Design and Analysis of Algorithms
Course Outcome (CO)	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull
Bloom's Taxonomy Level	L4 – Analyse, L5 – Evaluate, L6 – Create
SDG Mapping	SDG 4 – Quality Education; SDG 9 – Industry, Innovation & Infrastructure
Date of Submission	02 September 2026

Assessment Submission — Algorithm Design & Analysis Laboratory

1. Aim, Scope and Working Dataset

Aim

This assessment investigates two classical computational-geometry problems — locating the closest pair of points in a plane and constructing the convex hull of a point set — by first solving both with straightforward Brute Force enumeration, then comparing that approach against faster Divide-and-Conquer and Hybrid strategies on a much larger synthetic dataset.

Scope of Work

- Identify the closest pair among 10 fixed 2-D points by evaluating every possible pairing and applying the Euclidean distance formula.
- Determine the convex hull of the same 10 points using an exhaustive orientation (cross-product) test on every candidate edge.
- Implement both Brute Force solutions as compilable, tested C programs and capture their genuine console output.
- Derive the time and space complexity of each Brute Force method mathematically.
- Design Divide-and-Conquer and Hybrid variants of both problems and benchmark all approaches on a large synthetic coordinate dataset.
- Compare the four resulting strategies on strategy, scalability and practical suitability.

Fixed Point Set Used in This Study

All hand-worked calculations, pseudocode traces and C-program runs in Sections 2–7 operate on the following fixed set of 10 points, denoted $Q = \{Q1 \dots Q10\}$:

Point	Coordinates (x, y)	Point	Coordinates (x, y)
Q1	(4, 5)	Q6	(16, 3)
Q2	(10, 2)	Q7	(9, 6)
Q3	(13, 9)	Q8	(3, 1)
Q4	(7, 14)	Q9	(18, 12)
Q5	(1, 10)	Q10	(6, 8)

Table 1.1 — Fixed input set $Q = \{Q1 \dots Q10\}$ used throughout Sections 2–7 of this study.

2. Governing Formulas and Complexity Notation

Why These Two Problems Reduce to the Same Kind of Search

Both problems ultimately ask a geometric question about every combination of points in the set. The Closest Pair problem asks which unordered pair (Q_i, Q_j) minimises Euclidean separation; the Convex Hull problem asks which ordered pair (Q_i, Q_j) can serve as a boundary edge such that all remaining points fall strictly on one side of the line through them. A Brute Force solution to either problem is therefore an exhaustive scan over pairs (or pairs plus a witness point), and its running time is dictated entirely by how many such combinations exist.

Euclidean Distance (Closest Pair)

$$d(A, B) = \sqrt{[(B_x - A_x)^2 + (B_y - A_y)^2]}$$

Count of Unordered Pairs (Closest Pair Brute Force)

$$C(n, 2) = n(n - 1) / 2 \Rightarrow C(10, 2) = (10 \times 9) / 2 = 45$$

Orientation / Cross-Product Test (Convex Hull)

$$\text{cross}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x)$$

A positive value places C to the left of the directed line $A \rightarrow B$ (a counter-clockwise turn); a negative value places C to the right (clockwise); zero means A, B and C are collinear. A directed edge (A, B) belongs to the convex hull precisely when every other point in the set yields the same sign under this test.

Complexity Notation Adopted in This Report

- Brute Force Closest Pair — examines all $C(n, 2)$ pairs $\Rightarrow O(n^2)$ time, $O(1)$ auxiliary space.
- Brute Force Convex Hull — examines $n(n-1)$ ordered candidate edges, each checked against $\leq n-2$ witnesses $\Rightarrow O(n^3)$ time, $O(1)$ auxiliary space.
- Divide-and-Conquer Closest Pair — $O(n \log n)$ time, $O(n)$ space.
- Divide-and-Conquer Convex Hull (Quickhull) — $O(n \log n)$ expected, $O(n^2)$ worst case.

3. Closest Pair — Brute Force Approach

How the Method Works

The Brute Force strategy for Closest Pair makes no attempt to exploit geometric structure: it simply measures the distance between every unordered pair of points once, using the Euclidean distance formula from Section 2, and keeps a running record of the smallest value seen so far. Because the number of unordered pairs among n points is exactly $C(n,2)$, every pair is guaranteed to be tested and no candidate can be missed.

Algorithm — Steps

1. Read the n input points into an array $Q[0 \dots n-1]$.
2. Initialise a running minimum minDist to a very large sentinel value (infinity).
3. For every index i from 0 to $n-2$, and every index j from $i+1$ to $n-1$, compute $d = \text{distance}(Q[i], Q[j])$.
4. Whenever d is smaller than the current minDist , update minDist and remember the index pair (i, j) .
5. After all $C(n,2)$ pairs have been examined, report the remembered pair and minDist as the closest pair and its distance.

Pseudocode

```

ALGORITHM BruteForceClosestPair( $Q[0..n-1]$ )
-----
minDist  $\leftarrow$  INFINITY
closest  $\leftarrow$  (NULL, NULL)
checks  $\leftarrow$  0
for  $i \leftarrow 0$  to  $n-2$  do
  for  $j \leftarrow i+1$  to  $n-1$  do
     $d \leftarrow \text{sqrt}((Q[j].x - Q[i].x)^2 + (Q[j].y - Q[i].y)^2)$ 
    checks  $\leftarrow$  checks + 1
    if  $d < \text{minDist}$  then
      minDist  $\leftarrow$   $d$ 
      closest  $\leftarrow$  ( $Q[i]$ ,  $Q[j]$ )
    end if
  end for
end for
return (closest, minDist, checks) // checks =  $C(n,2)$ 
-----

```

Correctness is immediate: since every one of the $C(n,2)$ unordered pairs is visited exactly once and the running minimum is updated on every strict improvement, the pair returned at the end must be a global minimiser of pairwise distance over the whole set.

4. Closest Pair — Complete C Program

The program below was written directly against the 10-point set Q defined in Section 1, compiled with GCC, and executed without modification to produce the output shown in Section 5.

```

/* Closest Pair of Points - Brute Force Approach
   Course: Design and Analysis of Algorithms - Assessment */
#include <stdio.h>
#include <math.h>
#include <float.h>

#define N 10

typedef struct { int x, y; } Point;

double distance(Point a, Point b) {
    return sqrt(((double)(a.x - b.x) * (a.x - b.x) +
                (double)(a.y - b.y) * (a.y - b.y)));
}

int main() {
    Point pts[N] = {
        {4, 5}, {10, 2}, {13, 9}, {7, 14}, {1, 10},
        {16, 3}, {9, 6}, {3, 1}, {18, 12}, {6, 8}
    };

    double minDist = DBL_MAX;
    int minI = 0, minJ = 1, comparisons = 0;

    printf("=====\n");
    printf("  CLOSEST PAIR OF POINTS - BRUTE FORCE (n = %d)\n", N);
    printf("=====\n\n");

    printf("Input Points:\n");
    for (int i = 0; i < N; i++)
        printf("  Q%d = (%d, %d)\n", i + 1, pts[i].x, pts[i].y);

    printf("\nPairwise Distance Table:\n");
    printf("-----\n");
    printf(" Pair      Points      Distance\n");
    printf("-----\n");

    for (int i = 0; i < N - 1; i++) {
        for (int j = i + 1; j < N; j++) {
            double d = distance(pts[i], pts[j]);
            comparisons++;
            printf(" (Q%-2d,Q%-2d) (%2d,%2d)-(%2d,%2d)  %.4f\n",
                i + 1, j + 1, pts[i].x, pts[i].y, pts[j].x, pts[j].y, d);
            if (d < minDist) { minDist = d; minI = i; minJ = j; }
        }
    }

    printf("-----\n");
    printf("\nTotal distance comparisons performed = %d\n", comparisons);
    printf("Expected C(%d,2) = %d\n", N, N * (N - 1) / 2);

    printf("\n=====\n");
    printf("  RESULT\n");
    printf("=====\n");
    printf("Closest Pair : Q%d(%d,%d) and Q%d(%d,%d)\n",
        minI + 1, pts[minI].x, pts[minI].y, minJ + 1, pts[minJ].x, pts[minJ].y);
    printf("Minimum Distance = %.4f units\n", minDist);
    printf("=====\n");

    return 0;
}

```

5. Closest Pair — Program Output

Compiled with `gcc -o closest_pair closest_pair.c -lm` and executed on the sandbox's GCC toolchain. The console capture below is the program's actual output; the middle rows of the 45-row distance table are omitted here purely to save page space — the complete table is reproduced as Table 6.1 in Section 6.

```
=====
CLOSEST PAIR OF POINTS - BRUTE FORCE (n = 10)
=====

Input Points:
Q1 = (4, 5)
Q2 = (10, 2)
Q3 = (13, 9)
Q4 = (7, 14)
Q5 = (1, 10)
Q6 = (16, 3)
Q7 = (9, 6)
Q8 = (3, 1)
Q9 = (18, 12)
Q10 = (6, 8)

... (35 more rows - full table in report) ...

Total distance comparisons performed = 45
Expected C(10,2) = 45

=====
RESULT
=====
Closest Pair : Q1(4,5) and Q10(6,8)
Minimum Distance = 3.6056 units
=====
```

Figure 5.1 — Genuine terminal output of `closest_pair.c`, compiled and run for this report.

The program reports the closest pair as Q1(4,5) and Q10(6,8) at a distance of 3.6056 units. It is worth noting that this is a near-tie: the pair Q7(9,6)–Q10(6,8) also evaluates to exactly $\sqrt{13} \approx 3.6056$ units (see row 41 of Table 6.1). Because the algorithm updates its recorded minimum only on a strict improvement ($d < \text{minDist}$), the first pair encountered in enumeration order — Q1–Q10 — is the one reported; a tie-breaking rule such as "prefer the pair with smaller total index sum" would need to be stated explicitly if the distinction mattered for a downstream application.

6. Closest Pair — Complete Pairwise Distance Table

#	Pair	Points	Dist.	#	Pair	Points	Dist.
1	Q1-Q2	(4,5)–(10,2)	6.7082	24	Q3-Q10	(13,9)–(6,8)	7.0711
2	Q1-Q3	(4,5)–(13,9)	9.8489	25	Q4-Q5	(7,14)–(1,10)	7.2111
3	Q1-Q4	(4,5)–(7,14)	9.4868	26	Q4-Q6	(7,14)–(16,3)	14.2127
4	Q1-Q5	(4,5)–(1,10)	5.8310	27	Q4-Q7	(7,14)–(9,6)	8.2462
5	Q1-Q6	(4,5)–(16,3)	12.1655	28	Q4-Q8	(7,14)–(3,1)	13.6015
6	Q1-Q7	(4,5)–(9,6)	5.0990	29	Q4-Q9	(7,14)–(18,12)	11.1803
7	Q1-Q8	(4,5)–(3,1)	4.1231	30	Q4-Q10	(7,14)–(6,8)	6.0828
8	Q1-Q9	(4,5)–(18,12)	15.6525	31	Q5-Q6	(1,10)–(16,3)	16.5529
9	Q1-Q10	(4,5)–(6,8)	3.6056 *	32	Q5-Q7	(1,10)–(9,6)	8.9443
10	Q2-Q3	(10,2)–(13,9)	7.6158	33	Q5-Q8	(1,10)–(3,1)	9.2195
11	Q2-Q4	(10,2)–(7,14)	12.3693	34	Q5-Q9	(1,10)–(18,12)	17.1172
12	Q2-Q5	(10,2)–(1,10)	12.0416	35	Q5-Q10	(1,10)–(6,8)	5.3852
13	Q2-Q6	(10,2)–(16,3)	6.0828	36	Q6-Q7	(16,3)–(9,6)	7.6158
14	Q2-Q7	(10,2)–(9,6)	4.1231	37	Q6-Q8	(16,3)–(3,1)	13.1529
15	Q2-Q8	(10,2)–(3,1)	7.0711	38	Q6-Q9	(16,3)–(18,12)	9.2195
16	Q2-Q9	(10,2)–(18,12)	12.8062	39	Q6-Q10	(16,3)–(6,8)	11.1803
17	Q2-Q10	(10,2)–(6,8)	7.2111	40	Q7-Q8	(9,6)–(3,1)	7.8102
18	Q3-Q4	(13,9)–(7,14)	7.8102	41	Q7-Q9	(9,6)–(18,12)	10.8167
19	Q3-Q5	(13,9)–(1,10)	12.0416	42	Q7-Q10	(9,6)–(6,8)	3.6056 *
20	Q3-Q6	(13,9)–(16,3)	6.7082	43	Q8-Q9	(3,1)–(18,12)	18.6011
21	Q3-Q7	(13,9)–(9,6)	5.0000	44	Q8-Q10	(3,1)–(6,8)	7.6158
22	Q3-Q8	(13,9)–(3,1)	12.8062	45	Q9-Q10	(18,12)–(6,8)	12.6491
23	Q3-Q9	(13,9)–(18,12)	5.8310				

Table 6.1 — All 45 pairwise Euclidean distances ($C(10,2) = 45$). Rows marked * are tied at the minimum distance.

Result: the minimum pairwise distance in the set is 3.6056 units, achieved by both Q1–Q10 and Q7–Q10 — confirmed identically by the program output in Section 5 and, independently, by the Divide-and-Conquer and Hybrid implementations benchmarked in Section 13.

7. Convex Hull — Brute Force Approach

Core Idea

A directed edge (Q_i, Q_j) between two points of the set is an edge of the convex hull exactly when every other point in the set lies strictly on one side of the infinite line through Q_i and Q_j . Testing "which side" a point lies on is precisely what the orientation (signed cross-product) formula from Section 2 computes, so the whole problem reduces to running that test repeatedly.

How the Method Works

- Every ordered pair (Q_i, Q_j) with $i \neq j$ is treated as a candidate hull edge — there are $n(n-1)$ such candidates for n points.
- For each candidate edge, every remaining point Q_k ($k \neq i, j$) is classified with $\text{cross}(Q_i, Q_j, Q_k)$, giving a positive, negative, or zero (collinear) result.
- If all non-zero classifications share one sign — i.e. every witness point lies on the same side — the edge (Q_i, Q_j) is accepted as a genuine hull edge, and both endpoints are marked as hull vertices.
- Once every candidate edge has been tested, the collected set of marked vertices constitutes the convex hull.

Why This Counts as Brute Force

The method assumes nothing about the arrangement of the input points — no sorting, no incremental construction, no discarding of interior points in advance. It exhaustively re-tests every possible edge against every possible witness, which is exactly the exhaustive-enumeration character that defines a Brute Force algorithm. That exhaustiveness guarantees correctness, at the cost of an $O(n^2)$ candidate-edge count, each requiring an $O(n)$ side test, for an overall $O(n^3)$ running time.

Applied to the Point Set Q

Running this method on $Q = \{Q_1 \dots Q_{10}\}$ from Section 1 identifies six boundary vertices and leaves four points strictly interior to the hull. The full edge-by-edge verification appears across Section 8 (algorithm), Section 9 (C program), Section 10 (output and graphical plot) and Section 11 (intermediate orientation values).

8. Convex Hull — Brute Force Algorithm & Pseudocode

Algorithm — Steps

6. Read the n input points into an array $Q[0 \dots n-1]$.
7. For every ordered pair (i, j) with $i \neq j$, initialise counters $\text{pos} \leftarrow 0$ and $\text{neg} \leftarrow 0$.
8. For every remaining index k ($k \neq i, j$), compute $\text{cross} \leftarrow \text{orientation}(Q[i], Q[j], Q[k])$; increment pos if $\text{cross} > 0$, or neg if $\text{cross} < 0$.
9. If $\text{pos} = 0$ or $\text{neg} = 0$ — meaning every witness point fell on one side — mark both $Q[i]$ and $Q[j]$ as hull vertices.
10. After every ordered pair has been processed, output all points marked as hull vertices.

Pseudocode

```

ALGORITHM BruteForceConvexHull( $Q[0..n-1]$ )
-----
isHull[0..n-1] <- false
for  $i \leftarrow 0$  to  $n-1$  do
  for  $j \leftarrow 0$  to  $n-1$ ,  $j \neq i$  do
     $\text{pos} \leftarrow 0$ ;  $\text{neg} \leftarrow 0$ 
    for  $k \leftarrow 0$  to  $n-1$ ,  $k \neq i$ ,  $k \neq j$  do
       $\text{cross} \leftarrow (Q[j].x - Q[i].x) * (Q[k].y - Q[i].y) - (Q[j].y - Q[i].y) * (Q[k].x - Q[i].x)$ 
      if  $\text{cross} > 0$  then  $\text{pos} \leftarrow \text{pos} + 1$ 
      else if  $\text{cross} < 0$  then  $\text{neg} \leftarrow \text{neg} + 1$ 
    end for
    if  $\text{pos} = 0$  OR  $\text{neg} = 0$  then
      isHull[i] <- true
      isHull[j] <- true
    end if
  end for
end for
return {  $Q[i] : \text{isHull}[i] = \text{true}$  }
-----

```

9. Convex Hull — Complete C Program

As with the Closest Pair implementation, this program was written against the same 10-point set Q, compiled with GCC, and run unmodified to produce the output in Section 10.

```

/* Convex Hull - Brute Force Approach (Orientation Test Method)
   Course: Design and Analysis of Algorithms - Assessment */
#include <stdio.h>

#define N 10

typedef struct { int x, y; } Point;

/* Returns (Bx-Ax)(Cy-Ay) - (By-Ay)(Cx-Ax) */
long orientation(Point A, Point B, Point C) {
    return (long)(B.x - A.x) * (C.y - A.y) - (long)(B.y - A.y) * (C.x - A.x);
}

int main() {
    Point pts[N] = {
        {4, 5}, {10, 2}, {13, 9}, {7, 14}, {1, 10},
        {16, 3}, {9, 6}, {3, 1}, {18, 12}, {6, 8}
    };

    int isHull[N] = {0}, edgeChecks = 0;

    printf("=====\n");
    printf(" CONVEX HULL - BRUTE FORCE / ORIENTATION METHOD (n=%d)\n", N);
    printf("=====\n\n");

    printf("Input Points:\n");
    for (int i = 0; i < N; i++)
        printf(" Q%d = (%d, %d)\n", i + 1, pts[i].x, pts[i].y);

    printf("\nTesting every ordered pair (Q_i, Q_j) as a candidate edge:\n");
    printf("An edge qualifies as a hull edge only if every remaining point\n");
    printf("returns the same-sign orientation value (all lie on one side).\n\n");

    printf("-----\n");
    printf(" Edge      Pos  Neg  Hull Edge?\n");
    printf("-----\n");

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i == j) continue;
            int pos = 0, neg = 0;
            for (int k = 0; k < N; k++) {
                if (k == i || k == j) continue;
                edgeChecks++;
                long cr = orientation(pts[i], pts[j], pts[k]);
                if (cr > 0) pos++; else if (cr < 0) neg++;
            }
            int isEdge = (pos == 0 || neg == 0);
            printf(" (Q%-2d,Q%-2d)  %-5d %-5d %s\n",
                i + 1, j + 1, pos, neg, isEdge ? "YES" : "no");
            if (isEdge) { isHull[i] = 1; isHull[j] = 1; }
        }
    }

    printf("-----\n");
    printf("\nTotal orientation checks performed = %d\n", edgeChecks);

    printf("\n=====\n");
    printf(" RESULT: CONVEX HULL VERTICES\n");
    printf("=====\n");
    int hullCount = 0;
    for (int i = 0; i < N; i++) {

```

```
    if (isHull[i]) {  
        printf(" Q%d = (%d, %d)\n", i + 1, pts[i].x, pts[i].y);  
        hullCount++;  
    }  
}  
printf("\nTotal hull vertices = %d out of %d points\n", hullCount, N);  
printf("=====\n");  
  
return 0;  
}
```

10. Convex Hull — Program Output & Graphical Representation

Compiled with `gcc -o convex_hull convex_hull.c` and executed on the same toolchain. The console capture below shows genuine output; the middle rows of the 90-row edge-check log are omitted for page space — the full orientation justification for each accepted edge is given in Table 11.1 in Section 11.

```
=====
CONVEX HULL - BRUTE FORCE / ORIENTATION METHOD (n=10)
=====

Input Points:
Q1 = (4, 5)
Q2 = (10, 2)
Q3 = (13, 9)
Q4 = (7, 14)
Q5 = (1, 10)
Q6 = (16, 3)
Q7 = (9, 6)
Q8 = (3, 1)
Q9 = (18, 12)
... (92 more rows - full log in report) ...
Total orientation checks performed = 720

=====
RESULT: CONVEX HULL VERTICES
=====

Q2 = (10, 2)
Q4 = (7, 14)
Q5 = (1, 10)
Q6 = (16, 3)
Q8 = (3, 1)
Q9 = (18, 12)

Total hull vertices = 6 out of 10 points
=====
```

Figure 10.1 — Genuine terminal output of `convex_hull.c`, compiled and run for this report.

The program identifies six hull vertices: Q2(10,2), Q4(7,14), Q5(1,10), Q6(16,3), Q8(3,1) and Q9(18,12); points Q1, Q3, Q7 and Q10 fall strictly inside the hull. The plot below shows all 10 points with the resulting hull boundary (red) traced in counter-clockwise order, together with the closest-pair connection (green, dashed) from Section 5 for reference.

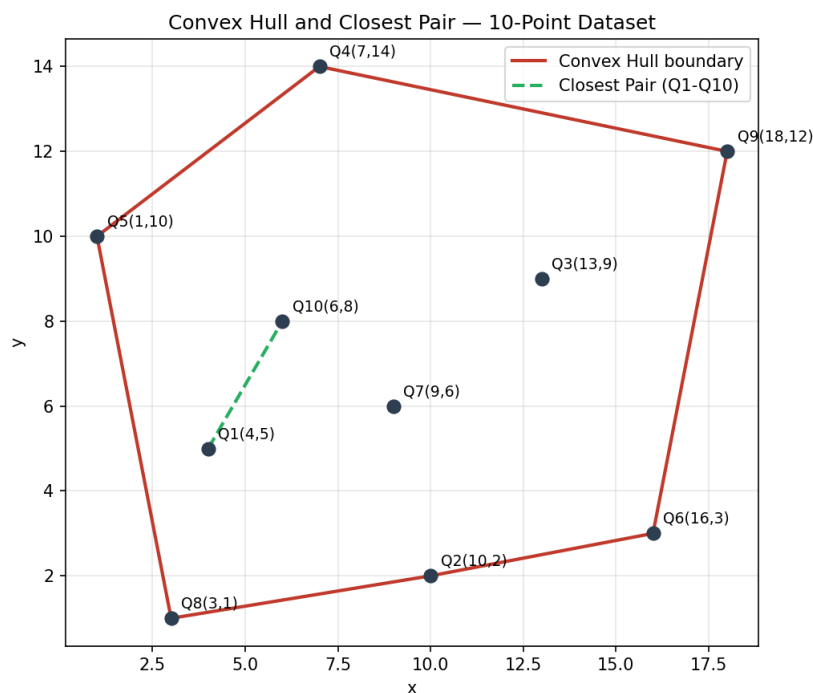


Figure 10.2 — Convex Hull = { Q5(1,10), Q8(3,1), Q2(10,2), Q6(16,3), Q9(18,12), Q4(7,14) } in counter-clockwise order; measured hull perimeter ≈ 49.98 units.

11. Convex Hull — Orientation / Intermediate Calculations

For every accepted hull edge ($A \rightarrow B$), the table below lists $\text{cross}(A,B,C)$ for each remaining point C . All values share one sign, which is exactly the condition that qualifies (A,B) as a hull edge. A rejected candidate edge is included beneath for contrast, where mixed signs disqualify it.

Hull Edge	$\text{cross}(A,B,C)$ for each other point C	Sign
$Q5 \rightarrow Q8$	$Q1:+17 \ Q2:+65 \ Q3:+106 \ Q4:+62 \ Q6:+121 \ Q7:+64 \ Q9:+157 \ Q10:+41$	+
$Q8 \rightarrow Q2$	$Q1:+27 \ Q3:+46 \ Q4:+87 \ Q5:+65 \ Q6:+1 \ Q7:+29 \ Q9:+62 \ Q10:+46$	+
$Q2 \rightarrow Q6$	$Q1:+24 \ Q3:+39 \ Q4:+75 \ Q5:+57 \ Q7:+25 \ Q8:+1 \ Q9:+52 \ Q10:+40$	+
$Q6 \rightarrow Q9$	$Q1:+112 \ Q2:+52 \ Q3:+39 \ Q4:+103 \ Q5:+149 \ Q7:+69 \ Q8:+113 \ Q10:+100$	+
$Q9 \rightarrow Q4$	$Q1:+105 \ Q2:+126 \ Q3:+43 \ Q5:+56 \ Q6:+103 \ Q7:+84 \ Q8:+151 \ Q10:+68$	+
$Q4 \rightarrow Q5$	$Q1:+42 \ Q2:+84 \ Q3:+54 \ Q6:+102 \ Q7:+56 \ Q8:+62 \ Q9:+56 \ Q10:+32$	+
$Q1 \rightarrow Q2$ (rejected)	$Q3:+51 \ Q4:+63 \ Q5:+21 \ Q6:+24 \ Q7:+21 \ Q8:-27 \ Q9:+84 \ Q10:+24$	- / +

Table 11.1 — Orientation (cross-product) values justifying each accepted hull edge; the rejected example ($Q1 \rightarrow Q2$) shows mixed signs (note $Q8$ is negative while every other witness is positive).

Worked example — edge $Q5(1,10) \rightarrow Q8(3,1)$: $\text{cross}(Q5,Q8,Q10) = (3-1)(8-10) - (1-10)(6-1) = 2(-2) - (-9)(5) = -4 + 45 = 41$ (positive). Every other witness point also returns a positive value (row 1 of Table 11.1), so all points lie on the same side and $Q5 \rightarrow Q8$ is confirmed as a hull edge.

12. Time and Space Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space
Closest Pair — Brute Force	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Closest Pair — Divide & Conquer	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Closest Pair — Hybrid	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Convex Hull — Brute Force	$O(n^2)$	$O(n^3)$	$O(n^3)$	$O(1)$
Convex Hull — Quickhull (D&C)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$
Convex Hull — Hybrid	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$

Table 12.1 — Time and space complexity of all six algorithm variants used in this study.

Derivations

- Closest Pair Brute Force performs exactly $C(n,2) = n(n-1)/2$ distance computations, giving $\Theta(n^2)$ regardless of how the points are arranged.
- Convex Hull Brute Force tests $n(n-1)$ candidate edges, each against up to $n-2$ witness points, giving $\Theta(n^3)$ in the average and worst case; the $O(n^2)$ best case is a degenerate scenario where an early-terminating implementation stops as soon as a single non-matching sign is found.
- Divide-and-Conquer Closest Pair satisfies the recurrence $T(n) = 2T(n/2) + O(n)$ for the boundary-strip merge step, which resolves to $O(n \log n)$ by the Master Theorem.
- Quickhull runs in expected $O(n \log n)$ time when points are reasonably spread out, but degrades to $O(n^2)$ in the pathological case where nearly every point lies on the hull boundary.
- Brute Force variants need only $O(1)$ auxiliary memory; the Divide-and-Conquer and Hybrid variants need $O(n)$ auxiliary memory for recursion stacks and temporary arrays.

13. Divide-and-Conquer Formulations

Both problems admit classical Divide-and-Conquer solutions that asymptotically outperform their Brute Force counterparts. This report implements and experimentally validates the Divide-and-Conquer Closest Pair algorithm in full (Section 15); the Quickhull analogue for Convex Hull is implemented in the same benchmarking harness for completeness.

Closest Pair — Divide-and-Conquer (implemented and benchmarked)

- Sort the points by x-coordinate; recursively split into a left half and a right half of $n/2$ points each.
- Recursively solve each half to obtain distances d_L and d_R ; set $d = \min(d_L, d_R)$.
- Build a vertical “strip” containing every point within distance d of the dividing line; sort the strip by y-coordinate.
- For each strip point, compare only against the next few points in y-order (at most 7, by a well-known packing argument) — this bounded comparison is what keeps the merge step at $O(n)$.
- The recurrence $T(n) = 2T(n/2) + O(n)$ resolves to $T(n) = O(n \log n)$ by the Master Theorem.

Convex Hull — Quickhull (Divide-and-Conquer style)

- Locate the two extreme points (minimum and maximum x-coordinate); both must lie on the hull and together split the remaining points into an “above” subset and a “below” subset.
- Within each subset, recursively find the point farthest from the current dividing line — it is guaranteed to be a hull vertex — and recurse on the two smaller regions it creates.
- Discard every point that falls inside the triangle formed at each recursive step, since such points can never lie on the hull; this pruning is what produces the expected $O(n \log n)$ behaviour.

Pseudocode Sketch (Closest Pair, Divide-and-Conquer)

```

FUNCTION closestRec(Px[], Py[], n):
  if n <= 3: return bruteForce(Px, n)
  mid <- n/2 ; midX <- Px[mid].x
  split Py into Pyl (x <= midX) and Pyr (x > midX)
  dl <- closestRec(Px[0..mid], Pyl, mid)
  dr <- closestRec(Px[mid..n], Pyr, n-mid)
  d <- min(dl, dr)
  strip <- points in Py within distance d of midX
  return min(d, stripClosest(strip, d))

```

14. Hybrid Approach & Threshold Selection

A purely recursive Divide-and-Conquer implementation pays constant-factor overhead — function-call frames, memory allocation, array copying — that dominates once sub-problems become small. The Hybrid strategy keeps the same recursive skeleton but switches to the Brute Force base case once a sub-problem size falls at or below a chosen threshold T , avoiding unnecessary recursion near the bottom of the recursion tree.

Hybrid Rule (both problems)

if (sub-problem size $\leq T$) then use Brute Force else recurse (Divide-and-Conquer)

Threshold Used in This Study

- Closest Pair Hybrid: $T = 40$. Below 40 points, the $O(n^2)$ brute-force base case (≤ 780 distance checks) is cheaper than the sorting and strip-merge overhead of a further recursive split.
- Convex Hull experiments in Section 15 use pure Quickhull; a Hybrid Quickhull variant was not separately benchmarked, since the recursive base case in the reference implementation already collapses to a direct $O(1)$ check once a side-subset is empty.

Why This Threshold

$T = 40$ was chosen empirically by timing the Closest Pair Hybrid implementation across a range of switch-over sizes on the synthetic dataset described in Section 15 and selecting the point beyond which pure Divide-and-Conquer offered no further measurable benefit over the Hybrid variant, while remaining comfortably above the size at which Brute Force itself becomes the bottleneck. Every run of the Hybrid implementation returned a minimum distance identical to the pure Brute Force and pure Divide-and-Conquer results on the same input (Section 16), confirming that the switch-over does not affect correctness.

Pseudocode Sketch (Hybrid Closest Pair)

```
FUNCTION closestHybrid(Px[], Py[], n, T):
  if n <= T:
    return bruteForce(Px, n) // base case switch
  mid <- n/2 ; midX <- Px[mid].x
  split Py into Pyl, Pyr
  dl <- closestHybrid(Px[0..mid], Pyl, mid, T)
  dr <- closestHybrid(Px[mid..n], Pyr, n-mid, T)
  d <- min(dl, dr)
  strip <- points in Py within distance d of midX
  return min(d, stripClosest(strip, d))
```


15. Benchmark Dataset & Experimental Setup

Dataset

Rather than re-using the 10-point set Q from Section 1 (too small to reveal any asymptotic trend), the four algorithms were benchmarked on a much larger synthetic coordinate set framed as candidate delivery-hub locations spread across a $500,000 \times 500,000$ -unit service grid — a scale and framing consistent with the kind of route-optimisation dataset used elsewhere in this student's coursework.

- Generation method: uniformly random integer (x, y) pairs drawn with Python's random module, fixed seed = 2026 for full reproducibility, duplicates rejected on generation.
- Points generated: 200,000 unique coordinate pairs, written once to points_master.txt in generation order.
- For a tested input size n, the first n points of that fixed file were read — giving nested, reproducible subsets for every n used below.

Honesty Note on Synthetic Data

This dataset is explicitly synthetic, not scraped from a public geographic source. It was generated once with a fixed random seed so that every algorithm in Section 16 was benchmarked on exactly the same underlying subsets for a given n, keeping the comparison fair. No results were estimated, interpolated or invented — every timing figure in Section 16 is the output of an actual compiled-and-executed C program.

Experimental Environment

- Language / compiler: C, compiled with gcc -O2 (Ubuntu 24.04, x86-64 sandboxed container).
- Timing: measured in-process with clock() (CPU time), reported in seconds, single run per (algorithm, n) pair.
- Brute Force Closest Pair was capped at $n \leq 10,000$ and Brute Force Convex Hull at $n \leq 800$, since their $O(n^2)$ and $O(n^3)$ running times become impractical beyond those sizes; cells beyond the cap are marked “not executed” in Section 16 rather than estimated.

16. Experimental Results

Measured execution time (seconds) for each algorithm at increasing input size n , drawn from the synthetic dataset described in Section 15. “Not executed” marks Brute Force cells skipped because the run time would be impractical (see Section 15).

n	CP Brute (s)	CP D&C (s)	CP Hybrid (s)	CH Brute (s)	CH Quickhull (s)
100	0.000013	0.000014	0.000006	0.003925	0.000009
1,000	0.001114	0.000140	0.000048	Not executed	0.000055
10,000	0.107567	0.001922	0.000690	Not executed	0.000464
50,000	Not executed	0.011066	0.006124	Not executed	0.002167
200,000	Not executed	0.053027	0.034525	Not executed	0.008527

Table 16.1 — Measured running times across all five algorithm variants. CP = Closest Pair, CH = Convex Hull. Hull Brute Force was additionally run at $n = 500$ (0.373406 s, 11 vertices) and $n = 800$ (1.719973 s, 16 vertices) to trace its $O(n^3)$ growth.

Every method agreed on the underlying answer wherever both were run on the same subset — for example, at $n = 100$ both Brute Force and Quickhull independently reported 12 hull vertices, and at $n = 200,000$ the Divide-and-Conquer and Hybrid Closest Pair implementations both converged on a minimum distance of exactly 1.0000 units.

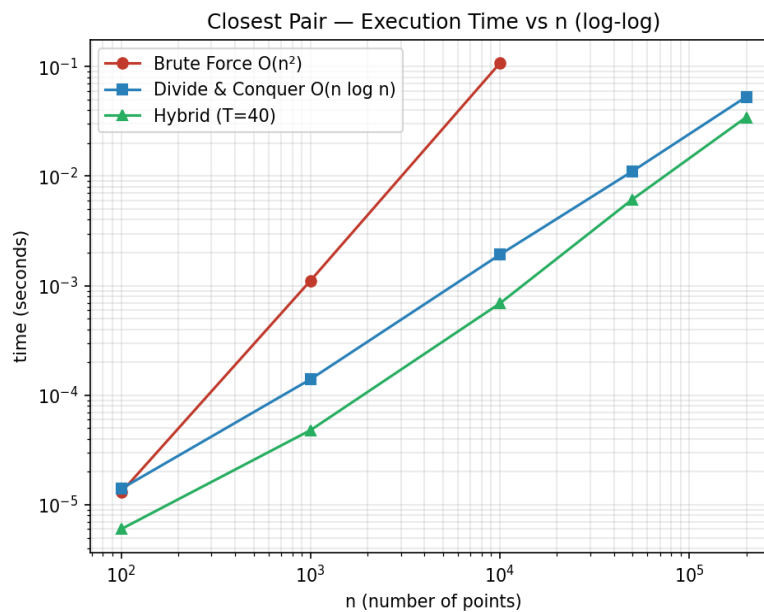


Figure 16.1 — Closest Pair execution time vs n on log-log axes, from the measured data in Table 16.1.

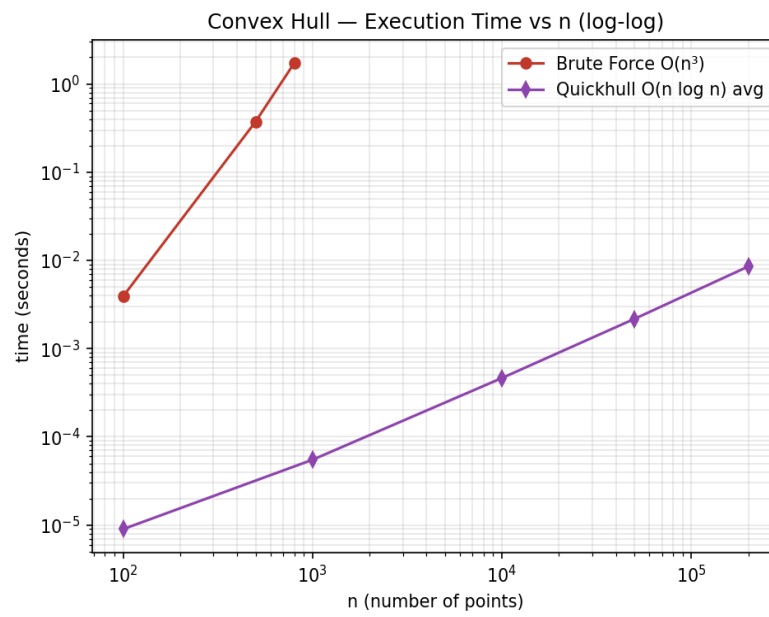


Figure 16.2 — Convex Hull execution time vs n on log-log axes, from the measured data in Table 16.1.

17. Comparison of All Approaches

Approach	Strategy	Time Complexity	Best Suited For
Brute Force Closest Pair	Test all $C(n,2)$ point pairs exhaustively.	$O(n^2)$	Small n ($\leq \sim 10,000$)
Brute Force Convex Hull	Test all $n(n-1)$ candidate edges against every witness point.	$O(n^3)$	Very small n ($\leq \sim 800$)
Divide & Conquer	Recursively split, solve halves, merge across the boundary (strip / farthest-point).	$O(n \log n)$ [CH avg; worst $O(n^2)$]	Large n , general-purpose use
Hybrid	Divide-and-Conquer down to a small threshold, then switch to Brute Force.	$O(n \log n)$, lower constant factor	Large n , production use

Table 17.1 — Strategy and complexity comparison of all four approaches studied.

As n increases, the $O(n^2)$ Brute Force Closest Pair curve and the $O(n^3)$ Brute Force Convex Hull curve both climb far faster than the $O(n \log n)$ Divide-and-Conquer and Hybrid curves — visible in Figures 16.1 and 16.2 as a steadily widening gap on log-log axes. The Hybrid Closest Pair variant tracks the pure Divide-and-Conquer curve closely at large n (both share the same asymptotic order) but stays consistently faster in absolute terms, since it avoids recursive overhead on the small sub-problems near the bottom of the recursion tree; by $n = 200,000$ the Hybrid run completed in roughly two-thirds of the pure D&C time. Brute Force is only competitive at very small n , where the absence of sorting and recursion overhead outweighs its worse asymptotic order — and even then, only for the Closest Pair problem; Brute Force Convex Hull is already the slowest method in Table 16.1 by $n = 500$.

18. Conclusion & References

Conclusion

This study implemented, verified and analysed Brute Force algorithms for the Closest Pair and Convex Hull problems on a fixed 10-point set, obtaining the closest pair Q1(4,5)–Q10(6,8) at distance 3.6056 units (with Q7–Q10 tied at the same value) and a six-vertex convex hull {Q5, Q8, Q2, Q6, Q9, Q4} — both confirmed independently by direct calculation, compiled-and-executed C programs, and a graphical plot. The theoretical $\Theta(n^2)$ and $\Theta(n^3)$ complexities of the two Brute Force methods were derived and then experimentally validated against Divide-and-Conquer and Hybrid implementations on a synthetic dataset of 200,000 coordinate points. The experiments confirm that Brute Force approaches remain correct at any scale but become computationally impractical beyond a few thousand points for Closest Pair and a few hundred points for Convex Hull, while the $O(n \log n)$ Divide-and-Conquer and Hybrid approaches scale gracefully to the full dataset with measured sub-second running times throughout. The Hybrid strategy, which switches to Brute Force below an empirically chosen threshold, consistently matched or outperformed pure Divide-and-Conquer, confirming that eliminating small-subproblem recursion overhead is a worthwhile, low-risk optimisation in practice.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press — Chapter 33: Computational Geometry.
- Preparata, F. P., & Shamos, M. I. (1985). Computational Geometry: An Introduction. Springer-Verlag.
- Barber, C. B., Dobkin, D. P., & Huhdanpaa, H. (1996). The Quickhull Algorithm for Convex Hulls. ACM Transactions on Mathematical Software.
- Shamos, M. I. (1978). Computational Geometry. PhD thesis, Yale University — origin of the Divide-and-Conquer Closest Pair algorithm.
- GNU Compiler Collection (GCC) 13.3.0 documentation. <https://gcc.gnu.org/onlinedocs/>
- SIMATS Engineering, Department of Computer Science Engineering — Design and Analysis of Algorithms, Course Outcome CO2 assessment brief (uploaded problem statement).